

CS 355 - Topic 8: Observer Pattern

Dates: March 18, 2026 and March 23, 2026

March 18:

Patterns in Programming

- In this class we cover two patterns:
 - asynchronous programming with callbacks
 - observer pattern
- No pattern is inherently better than another, it is your responsibility to identify when and where to use a specific programming pattern
- As computer science students it is our responsibility to master as many programming patterns as possible
- The async. and observer patterns are actually compliments, meaning where one is weak the other is strong

Pros and Cons of Asynchronous Programming with Callbacks:

- Pros:
 - Multitasking without direct thread access
 - Models long running non CPU bound tasks
 - Callbacks are atomic
- Cons:
 - Singular nature, multiple is nontrivial
 - Expectation that the "event" will eventually occur and anything else is considered an error state

Observer Pattern Overview

- Observer pattern fires multiple times, unlike the asynchronous pattern
- Subject: Maintains a list of observers (listener functions) per event type. When an event occurs, it notifies all listeners registered to that event.
- Observer: A listener function registered with the subject for a specific event. When the subject triggers that event, the listener function is called automatically.

Observer Pattern Sample Code

```
const readline = require("readline");

// start of subject
const rl = readline.createInterface({input: process.stdin, output:
process.stdout});
// end of subject

// start of observer
```

```
rl.on('line', (input) => {
  console.log(`Received: ${input}`);
});
// end of observer

console.log("Enter any input & press enter to fire line event");
```

- Subject (rl) — a readline interface that listens to terminal input. It knows how to detect when you press Enter and can emit events like 'line'. It holds the list of listeners registered to those events.
- Observer (rl.on('line', ...)) — registers an anonymous listener function to the 'line' event on rl. It just sits idle until the subject fires.
- When you press Enter — rl detects a completed line of input and automatically calls every listener registered to 'line', passing in what you typed as input. The listener then prints **Received: <what you typed>** to the console.

Observer Pattern Implementation in Node.js

- Node.js has a built-in **EventEmitter** class that implements the Observer pattern
- Any object that extends **EventEmitter** acts as the **subject**
- Subject: calls **emit("eventName", [...data])** to trigger an event and pass data to all registered listeners
- Observer: calls **on("eventName", callbackFn)** to register a listener function to a named event — the callback runs every time that event is emitted
- Internally, **EventEmitter** maintains a **listeners object** (not array) that maps each event name to an array of its registered listener functions
 - e.g. **{ "line": [fn1, fn2], "close": [fn3] }**
 - When **emit** is called, it looks up the event name and calls each function in its array in order

Demo 2 - Subject Focus

- Subject and observer are almost always in separate files
- Subject file tends to be a class file
- DayEmitter.js - Subject Class

```
const EventEmitter = require('events');
class DayEmitter extends EventEmitter {
  constructor(update_time = 240) {
    super();
    this.day = new Date();
    this.update_time = update_time;
  }
  start(){
    this.day.setDate(this.day.getDate() + 1);
    let mm = `${(this.day.getMonth() + 1 + "").padStart(2, "0")}`;
    let dd = `${(this.day.getDate() + "").padStart(2, "0")}`;
    this.emit('newday', {mm_dd: `${mm}/${dd}`});
    this.sleep();
  }
  sleep(){
```

```

        setTimeout(() => this.start(), this.update_time);
    }
}
module.exports = DayEmitter;

```

- index.js - Observer Class

```

const DayEmitter = require("./modules/DayEmitter");
const day_emitter = new DayEmitter();
day_emitter.on("newday", function({mm_dd}){
    process.stdout.cursorTo(0, 0);
    process.stdout.clearLine();
    process.stdout.write(mm_dd);
    process.stdout.cursorTo(0, 1);
});
console.clear();
day_emitter.start();

```

- How these two files work together:
 - `index.js` imports `DayEmitter`, creates an instance, and registers a listener via `.on("newday", ...)` before calling `.start()`. The order matters here, because the observer must be registered first or it will miss the first emit
 - When `.start()` is called, it advances the date by 1 day, formats it as `MM/DD`, then fires `emit("newday", { mm_dd })` — this is what triggers the listener in `index.js` and causes it to overwrite the terminal output with the new date
 - After emitting, `start()` calls `sleep()`, which uses `setTimeout` to call `start()` again after `update_time` ms (default 240ms) — creating a continuous loop that keeps emitting `newday` and updating the display
 - The observer uses `process.stdout` directly instead of `console.log` so it can overwrite the same line in the terminal (cursor is reset to position 0,0 each time) rather than printing a new line every tick

March 23:

Observer Pattern Review

- One of the many design patterns in computer science
- Allows multiple events naturally
- Allows 0 to be a valid state which is why it is passive in nature

BirthdayEmitter.js:

```

const EventEmitter = require("events");
class BirthdayEmitter extends EventEmitter {
    constructor({birthdays, day_emitter}){
        super();
    }
}

```

```

    day_emitter.on("newday", ({mm_dd}) => {
      let month = Number.parseInt(`${mm_dd[0]}${mm_dd[1]}`);
      let day = Number.parseInt(`${mm_dd[3]}${mm_dd[4]}`);
      birthdays
        .filter(birthday => birthday.month === month && birthday.day ===
day)
        .forEach(birthday => this.emit("birthday", {birthday}));
    });
  }
}
module.exports = BirthdayEmitter;

```

- supporting index.js:

```

const birthdays = require("./data/birthdays.json");
const DayEmitter = require("./modules/DayEmitter");
const BirthdayEmitter = require("./modules/BirthdayEmitter");

const day_emitter = new DayEmitter();
const birthday_emitter = new BirthdayEmitter({birthdays, day_emitter});
let current_line = 1;

birthday_emitter.on("birthday", function({birthday}){
  process.stdout.cursorTo(0, current_line);
  current_line++;
  console.log(birthday);
});

console.clear();
day_emitter.start();

```

- Breakdown of the code:
 - `BirthdayEmitter.js` accepts a list of `birthdays` and a clock
 - It listens for the clocks `newday` events and will cross reference the emitted `mm_dd` with its list of `birthdays`
 - Each time the `mm_dd` matches up with any `birthday`, a birthday event is emitted
 - If multiple people share the same birthday then multiple events are emitted
 - We feed in `day_emitter` to `BirthdayEmitter`'s constructor so that they share the same clock
 - This creates a chain of observers: `day_emitter` emits `newday` → `BirthdayEmitter` listens and emits `birthday` → `index.js` listens and prints
 - This means when `DayEmitter` gets paused, the `BirthdayEmitter` isn't receiving any `newdays` and no `birthdays` can be printed to the terminal
 - `BirthdayEmitter` acts as both an observer (listening to `day_emitter`) and a subject (emitting `birthday` events to whoever is listening)
 - In `index.js`, the observer registers on `birthday_emitter` (not `day_emitter` directly) — it only cares about birthdays, not every single day tick

- `day_emitter.start()` is the single trigger that kicks off the entire chain — nothing runs until it's called

Tight Coupling

- Definition: If the subject doesn't exist then the observer doesn't exist
 - e.g. in the BirthdayEmitter demo, `BirthdayEmitter` is tightly coupled to `day_emitter` because it is directly passed in and referenced inside the constructor
- The pub/ is the solution — instead of observers referencing the subject directly, both communicate through a shared event bus, so neither depends on the other